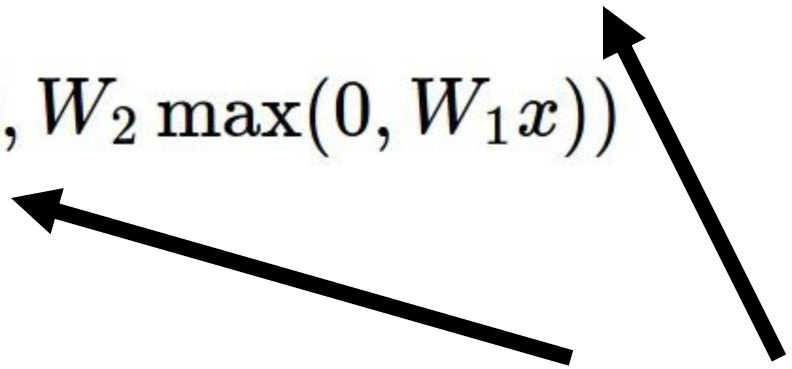
Neural networks

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

or 3-layer Neural Network

$$f = W_3 \max(0, W_2 \max(0, W_1 x))$$



also called "**Multilayer Perceptrons**" (MLPs)

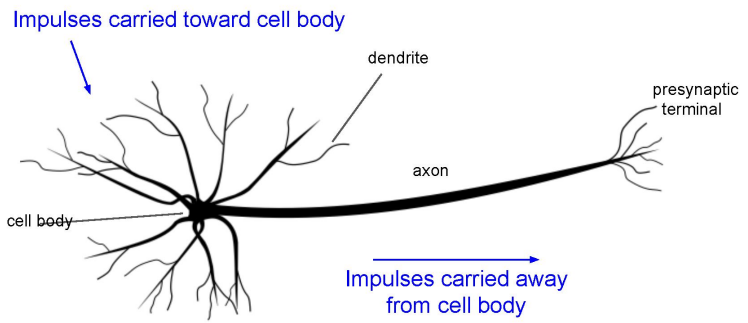
Adapted from Fei-Fei Li & Andrej Karpathy & Serena Leung

Neural networks

- Very coarse generalization of neural networks:
  - Linear functions chained together and separated by non-linearities (*activation functions*), e.g. "max"

$$f = W_3 \max(0, W_2 \max(0, W_1 x))$$

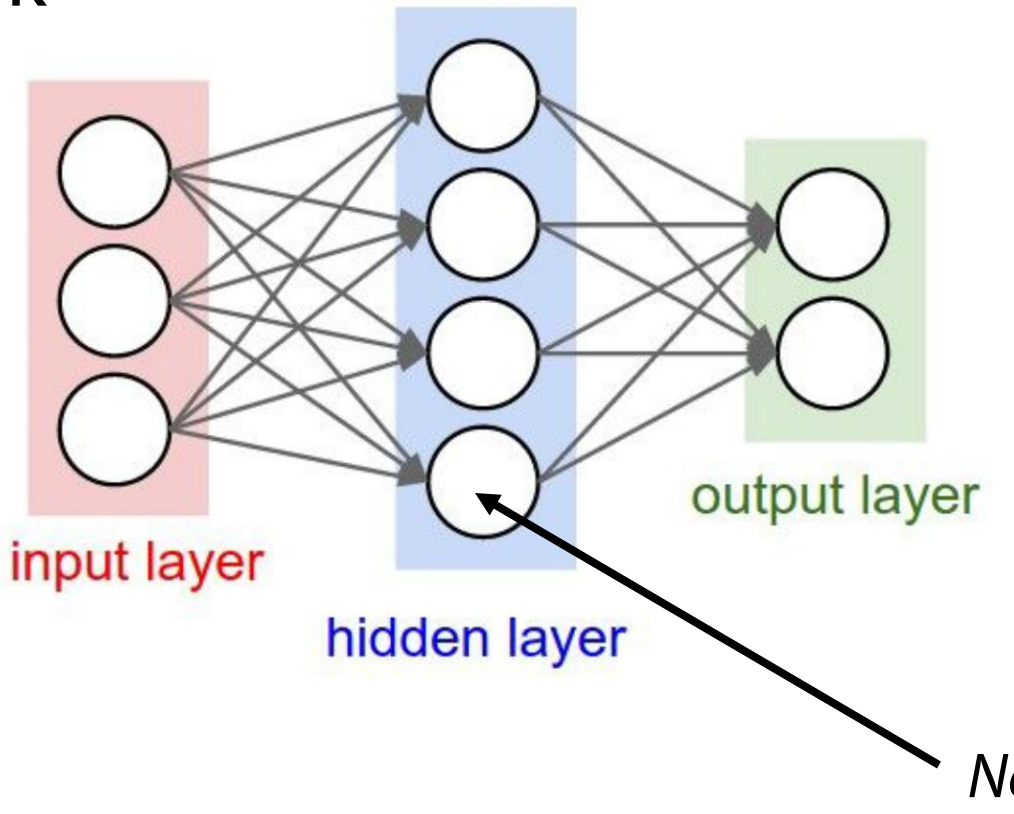
  - Why separate linear functions with non-linear functions?
  - *Very roughly* inspired by real neurons



This image by Felipe Perucho is licensed under CC-BY 3.0

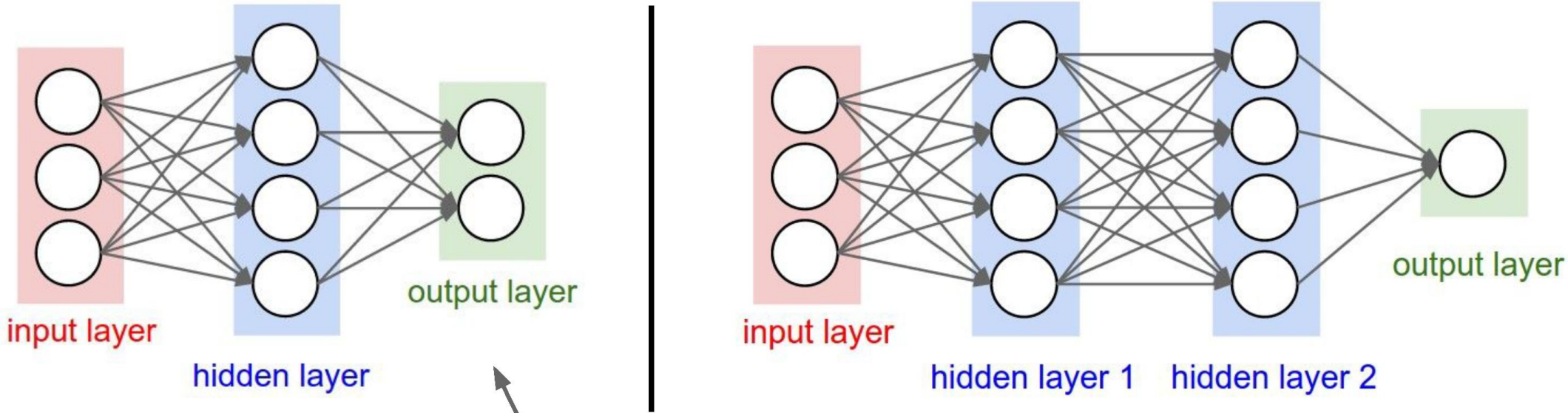
Neural network architecture

- Computation graph for a 2-layer neural network



Adapted from Fei-Fei Li & Andrej Karpathy & Serena Leung

Neural networks: architectures



"2-layer Neural Net", or
"1-hidden-layer Neural Net"

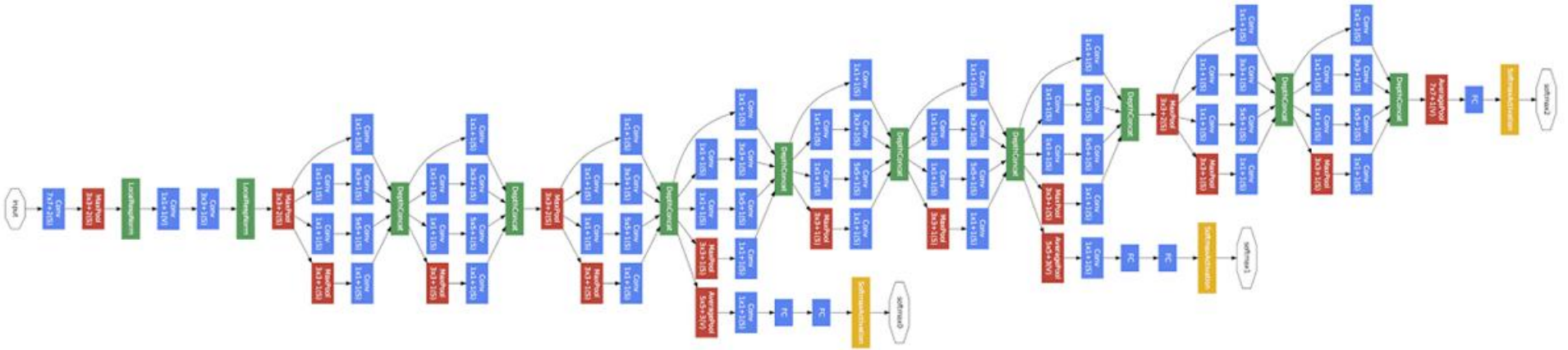
"3-layer Neural Net", or
"2-hidden-layer Neural Net"

"Fully-connected" layers

- **Deep** networks typically have many layers and potentially millions of parameters

Adapted from Fei-Fei Li & Andrej Karpathy & Serena Leung

Deep neural network

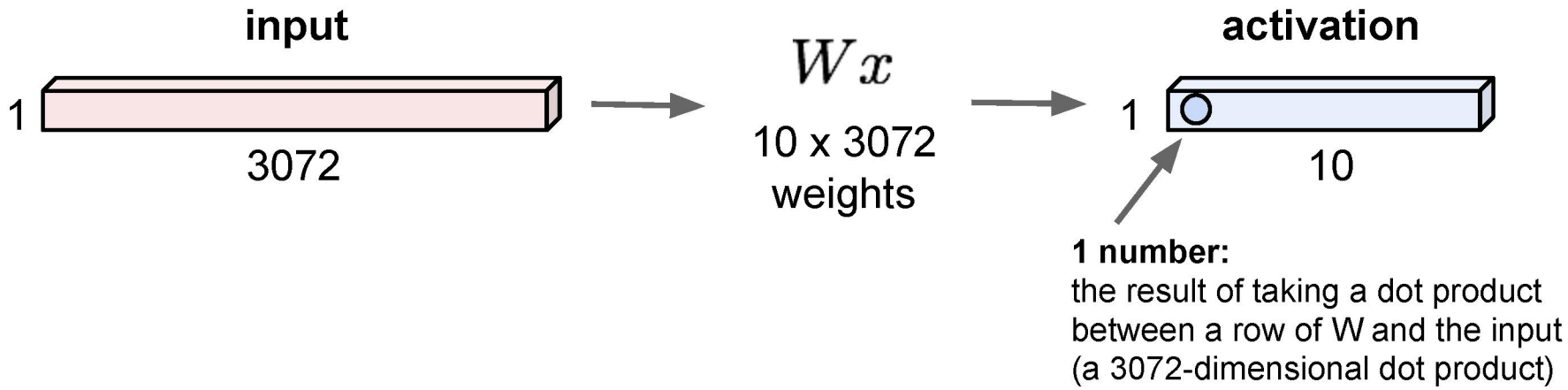


- *Inception* network (Szegedy et al, 2015)
- 22 layers

Adapted from Fei-Fei Li & Andrej Karpathy & Serena Leung

Fully Connected Layer

32x32x3 image -> stretch to 3072 x 1



- Just like a linear classifer – but in this case, just one layer of a larger *network*

Summary so far

- A classic neural network arranges neurons into fully-connected layers. This kind of neural network is also called a **Multi-Layer Perceptron**
- The **layer** abstraction enables efficient implementations of neural networks using vectorized operations like matrix multiplication

Optimizing parameters with gradient descent

- How do we find the best **W** and **b** parameters?
- In general: *gradient descent*
  1. Start with a guess of a good **W** and **b** (or randomly initialize them)
  2. Compute the loss function for this initial guess and the *gradient* of the loss function
  3. Step some distance in the negative gradient direction (direction of steepest descent)
  4. Repeat steps 2 & 3
- Note: efficiently performing step 2 for deep networks is called *backpropagation*